



INFORME DE GUÍA PRÁCTICA

1. Portada

Tema:	Árboles Binarios de Búsqueda (ABB): Recorridos, Conteo y Aplicaciones
Unidad de Organización Curricular:	Unidad 3 - Estructuras No Lineales
Nivel y Paralelo:	Tercero B
Alumno(s) participante(s):	Miranda Rolando
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

2. Informe de guía práctica

2.1. Objetivos

2.1.1. General

Implementar y aplicar Árboles Binarios de Búsqueda (ABB) en los lenguajes de programación C++ y Java, mediante la codificación de operaciones de inserción, recorrido, conteo de nodos y desarrollo de una aplicación práctica, con el fin de comprender su funcionamiento, estructura y utilidad en la resolución de problemas computacionales.

2.1.2. Específicos

- Implementar los recorridos preorden, inorden, postorden y BFS sobre un Árbol Binario de Búsqueda tanto en C++ como en Java.
- Desarrollar funciones para contar nodos totales y nodos hoja dentro de un ABB, visualizando su estructura de forma gráfica en consola.
- Construir una aplicación interactiva que modele un sistema web mediante un árbol binario, aplicando los distintos recorridos para simular funcionalidades reales.

2.2. Modalidad

Trabajo práctico individual / grupal — presencial y no presencial.

2.3. Tiempo de duración

Presencial: 2 horas

No presencial: 4 horas

2.4. Instrucciones

Se desarrollaron cinco ejercicios progresivos sobre Árboles Binarios de Búsqueda. Cada ejercicio amplía al anterior, incorporando nuevas funcionalidades. Los ejercicios fueron implementados en dos lenguajes: C++ y Java. Al finalizar cada implementación se tomó captura de la ejecución en consola.



2.5. Listado de equipos, materiales y recursos

Listado de equipos y materiales generales:

- Computador personal con compilador g++ (C++) y JDK (Java)
- IDE: Visual Studio Code / IntelliJ IDEA / NetBeans
- Terminal / Símbolo del sistema

TAC (Tecnologías para el Aprendizaje y Conocimiento):

- ☒ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☒ Inteligencia Artificial
- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Gamificación
- Otros: Compiladores en línea (Replit, OnlineGDB)

2.6. Actividades por desarrollar

1. Implementar un ABB con inserción y recorridos (Preorden, Inorden, Postorden, BFS).
2. Ampliar el ABB con nodos adicionales y volver a aplicar los recorridos.
3. Visualizar la estructura del árbol en consola con formato de árbol y contar nodos totales.
4. Extender el ejercicio 3 para contar únicamente los nodos hoja.
5. Construir un árbol de módulos de un Sistema Web e implementar un menú interactivo con los tres recorridos como funcionalidades del sistema.

2.7. Resultados obtenidos

A continuación se presentan los cinco ejercicios resueltos, mostrando el código fuente (C++ y Java) y las capturas de pantalla de la ejecución en consola.

Ejercicio 1 — Árbol Binario de Búsqueda: Recorridos básicos

Se construye un ABB con los valores {10, 5, 15, 2, 7, 12, 20}. Cada nodo almacena un entero y dos punteros (izquierdo y derecho); la función `insertar()` ubica cada valor comparándolo con la raíz y descendiendo por la rama que corresponde. Una vez construido el árbol, se aplican cuatro recorridos: **Preorden** (raíz → izq → der), **Inorden** (izq → raíz → der, produce la secuencia ordenada), **Postorden** (izq → der → raíz) y **BFS** (nivel a nivel, usando una cola auxiliar).

Implementación en C++:



```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4
5 struct Nodo {
6     int dato;
7     Nodo* izquierdo;
8     Nodo* derecho;
9 };
10
11 Nodo* crearNodo(int valor) {
12     Nodo* nuevo = new Nodo();
13     nuevo->dato = valor;
14     nuevo->izquierdo = NULL;
15     nuevo->derecho = NULL;
16     return nuevo;
17 }
18
19 Nodo* insertar(Nodo* raiz, int valor) {
20     if (raiz == NULL) return crearNodo(valor);
21     if (valor < raiz->dato) raiz->izquierdo = insertar(raiz->izquierdo, valor);
22     else if (valor > raiz->dato) raiz->derecho = insertar(raiz->derecho, valor);
23     return raiz;
24 }
25
26 void preorden(Nodo* raiz) {
27     if (raiz == NULL) return;
28     cout << raiz->dato << " ";
29     preorden(raiz->izquierdo);
30     preorden(raiz->derecho);
31 }
32
33 void inorden(Nodo* raiz) {
34     if (raiz == NULL) return;
35     inorden(raiz->izquierdo);
36     cout << raiz->dato << " ";
37     inorden(raiz->derecho);
38 }
39
40 void postorden(Nodo* raiz) {
41     if (raiz == NULL) return;
42     postorden(raiz->izquierdo);
43     postorden(raiz->derecho);
44     cout << raiz->dato << " ";
45 }
46
47 void bfs(Nodo* raiz) {
48     if (raiz == NULL) return;
49     queue<Nodo*> cola;
50     cola.push(raiz);
51     while (!cola.empty()) {
52         Nodo* actual = cola.front(); cola.pop();
53         cout << actual->dato << " ";
54         if (actual->izquierdo != NULL) cola.push(actual->izquierdo);
55         if (actual->derecho != NULL) cola.push(actual->derecho);
56     }
57 }
58
59 int main() {
60     Nodo* raiz = NULL;
61     raiz = insertar(raiz, 10);
```



```
62     insertar(raiz, 5); insertar(raiz, 15);
63     insertar(raiz, 2); insertar(raiz, 7);
64     insertar(raiz, 12); insertar(raiz, 20);
65     cout << "--- EJERCICIO 1 ---" << endl;
66     cout << "Preorden: "; preorden(raiz); cout << endl;
67     cout << "Inorden: "; inorden(raiz); cout << endl;
68     cout << "Postorden: "; postorden(raiz); cout << endl;
69     cout << "BFS: "; bfs(raiz); cout << endl;
70     return 0;
71 }
```

Listing 1: Ejercicio 1 — Recorridos ABB en C++

Implementación en Java:

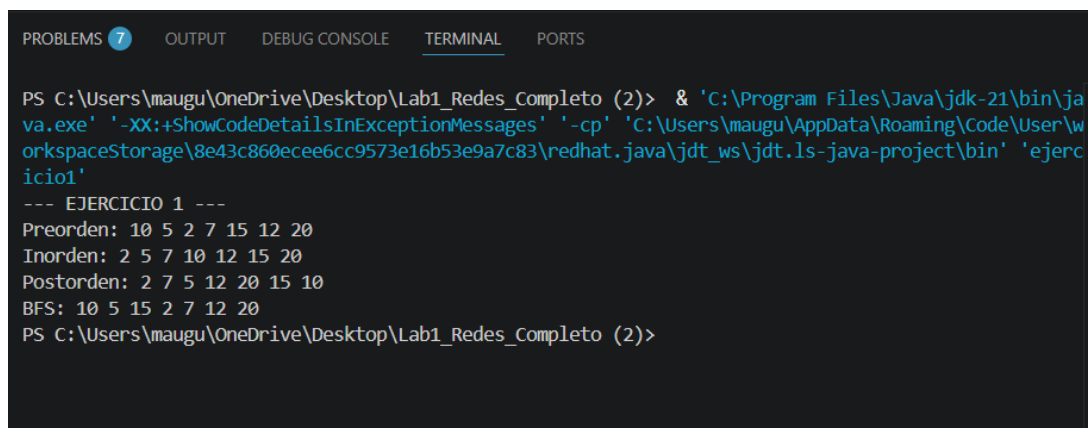
```
1  import java.util.LinkedList;
2  import java.util.Queue;
3
4  class Nodo {
5      int dato;
6      Nodo izquierdo, derecho;
7      public Nodo(int dato) { this.dato = dato; }
8  }
9
10 public class ejercicio1 {
11     public static Nodo insertar(Nodo raiz, int valor) {
12         if (raiz == null) return new Nodo(valor);
13         if (valor < raiz.dato) raiz.izquierdo = insertar(raiz.izquierdo, valor);
14         else if (valor > raiz.dato) raiz.derecho = insertar(raiz.derecho, valor);
15         return raiz;
16     }
17     public static void preorden(Nodo raiz) {
18         if (raiz == null) return;
19         System.out.print(raiz.dato + " ");
20         preorden(raiz.izquierdo); preorden(raiz.derecho);
21     }
22     public static void inorden(Nodo raiz) {
23         if (raiz == null) return;
24         inorden(raiz.izquierdo);
25         System.out.print(raiz.dato + " ");
26         inorden(raiz.derecho);
27     }
28     public static void postorden(Nodo raiz) {
29         if (raiz == null) return;
30         postorden(raiz.izquierdo); postorden(raiz.derecho);
31         System.out.print(raiz.dato + " ");
32     }
33     public static void bfs(Nodo raiz) {
34         if (raiz == null) return;
35         Queue<Nodo> cola = new LinkedList<>();
36         cola.add(raiz);
37         while (!cola.isEmpty()) {
38             Nodo actual = cola.poll();
39             System.out.print(actual.dato + " ");
40             if (actual.izquierdo != null) cola.add(actual.izquierdo);
41             if (actual.derecho != null) cola.add(actual.derecho);
42         }
43     }
44     public static void main(String[] args) {
45         Nodo raiz = null;
46         raiz = insertar(raiz, 10);
47         insertar(raiz, 5); insertar(raiz, 15);
48         insertar(raiz, 2); insertar(raiz, 7);
```



```
49     insertar(raiz, 12); insertar(raiz, 20);
50     System.out.println("--- EJERCICIO 1 ---");
51     System.out.print("Preorden: "); preorden(raiz); System.out.println();
52     System.out.print("Inorden: "); inorden(raiz); System.out.println();
53     System.out.print("Postorden: "); postorden(raiz); System.out.println();
54     System.out.print("BFS: "); bfs(raiz); System.out.println();
55 }
56 }
```

Listing 2: Ejercicio 1 — Recorridos ABB en Java

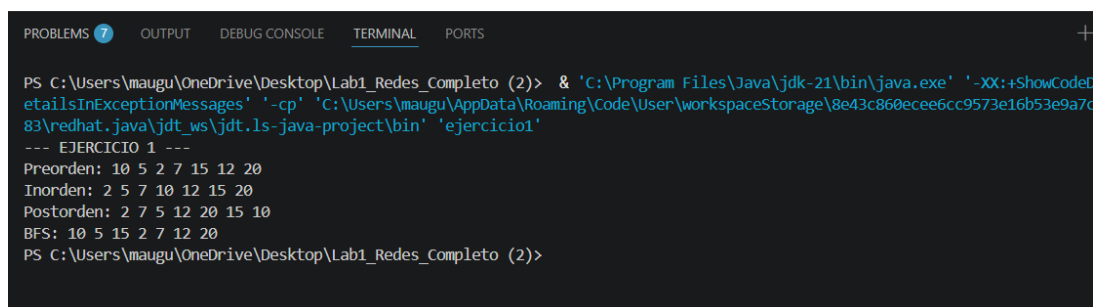
Capturas de ejecución — Ejercicio 1:



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'ejercicio1'
--- EJERCICIO 1 ---
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)>
```

Figura 1: Ejercicio 1 — Ejecución en C++: recorridos Preorden, Inorden, Postorden y BFS



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'ejercicio1'
--- EJERCICIO 1 ---
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)>
```

Figura 2: Ejercicio 1 — Ejecución en Java: recorridos Preorden, Inorden, Postorden y BFS

Ejercicio 2 — ABB Ampliado: Nuevos nodos y recorridos

Se parte del árbol construido en el Ejercicio 1 y se insertan cuatro nodos adicionales: {1, 3, 18, 25}. El árbol queda con 11 nodos en total; los nuevos valores se ubican automáticamente en la posición correcta gracias a la propiedad de orden del ABB. Se repiten los cuatro recorridos para observar cómo el resultado cambia al crecer el árbol.

Fragmento clave C++ — inserción adicional:

```
1 // Nodos del ejercicio 1
2 raiz = insertar(raiz, 10);
```



```
3 insertar(raiz, 5); insertar(raiz, 15);
4 insertar(raiz, 2); insertar(raiz, 7);
5 insertar(raiz, 12); insertar(raiz, 20);
6
7 // Nodos nuevos del ejercicio 2
8 insertar(raiz, 1); insertar(raiz, 3);
9 insertar(raiz, 18); insertar(raiz, 25);
10
11 cout << "--- EJERCICIO 2 (Modificado) ---" << endl;
12 cout << "Preorden: "; preorden(raiz); cout << endl;
13 cout << "Inorden: "; inorden(raiz); cout << endl;
14 cout << "Postorden: "; postorden(raiz); cout << endl;
15 cout << "BFS: "; bfs(raiz); cout << endl;
```

Listing 3: Ejercicio 2 — Inserción de nuevos nodos (C++)

Capturas de ejecución — Ejercicio 2:

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> cd "c:\Users\maugu\Downloads\" ; if ($?) { g++ ejercicio1.cpp
o ejercicio1 } ; if ($?) { .\ejercicio1 }
--- EJERCICIO 1 ---
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
PS C:\Users\maugu\Downloads>
```

Figura 3: Ejercicio 2 — Ejecución en C++: árbol ampliado con nodos {1, 3, 18, 25}

```
PS C:\Users\maugu\Downloads> cd "c:\Users\maugu\Downloads\" ; if ($?) { g++ ejercicio2.cpp -o ejercicio2 } ; if ($?) { .\ejercicio
2 }
--- EJERCICIO 2 (Modificado) ---
Preorden: 10 5 2 1 3 7 15 12 20 18 25
Inorden: 1 2 3 5 7 10 12 15 18 20 25
Postorden: 1 3 2 7 5 12 18 25 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 18 25
```

Figura 4: Ejercicio 2 — Ejecución en Java: árbol ampliado con nodos {1, 3, 18, 25}

Ejercicio 3 — Visualización estructurada y conteo de nodos

Se añaden dos funciones al ejercicio anterior. La función `imprimir()` recorre el árbol recursivamente e imprime cada nodo con prefijos gráficos (`|-->` para hijo izquierdo, `\-->` para el último hijo), logrando una representación visual de la jerarquía directamente en consola. La función `contarNodos()` suma 1 por cada nodo visitado de forma recursiva y retorna el total.

Función de impresión y conteo C++:

```
1 void imprimir(Nodo* raiz, string prefijo, bool esUltimo) {
2     if (raiz == NULL) return;
3     cout << prefijo << (esUltimo ? "\\--> " : "|--> ") << raiz->dato << endl;
4     string nuevoPrefijo = prefijo + (esUltimo ? " " : "| ");
5     bool izquierdoEsUltimo = (raiz->derecho == NULL);
6     imprimir(raiz->izquierdo, nuevoPrefijo, izquierdoEsUltimo);
7     imprimir(raiz->derecho, nuevoPrefijo, true);
8 }
9
```



```
10 int contarNodos(Nodo* raiz) {  
11     if (raiz == NULL) return 0;  
12     return 1 + contarNodos(raiz->izquierdo) + contarNodos(raiz->derecho);  
13 }
```

Listing 4: Ejercicio 3 — Visualización y conteo de nodos (C++)

Capturas de ejecución — Ejercicio 3:

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsIn  
ExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.java\j  
dt_ws\jdt.ls-java-project\bin' 'ejercicio2'  
--- EJERCICIO 2 (Modificado) ---  
Preorden: 10 5 2 1 3 7 15 12 20 18 25  
Inorden: 1 2 3 5 7 10 12 15 18 20 25  
Postorden: 1 3 2 7 5 12 18 25 20 15 10  
BFS: 10 5 15 2 7 12 20 1 3 18 25
```

Figura 5: Ejercicio 3 — Ejecución en C++: visualización jerárquica del árbol y conteo de nodos

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> cd "c:\Users\maugu\Downloads\" ; if ($?) { g++ ejercicio3.cpp -o ejerc  
icio3 } ; if ($?) { .\ejercicio3 }  
--- EJERCICIO 3 ---  
Arbol Estructurado:  
10  
|--> 5  
|   |--> 2  
|   |   |--> 1  
|   |   \--> 3  
|   \--> 7  
|--> 15  
|   |--> 12  
|   |   |--> 18  
|   |   \--> 25  
-----  
La cantidad total de nodos es: 11
```

Figura 6: Ejercicio 3 — Ejecución en Java: visualización jerárquica del árbol y conteo de nodos

Ejercicio 4 — Conteo de nodos hoja

Se extiende el Ejercicio 3 añadiendo la función `contarHojas()`, que recorre el árbol recursivamente e incrementa un contador solo cuando el nodo visitado no tiene hijos (ambos punteros nulos). Un nodo hoja representa un dato final sin ramificaciones; conocer su cantidad es útil para analizar la densidad y el balance del árbol. La visualización jerárquica del ejercicio anterior se conserva.

Función `contarHojas` en Java:

```
1 public static int contarHojas(Nodo raiz) {  
2     if (raiz == null) return 0;  
3     if (raiz.izquierdo == null && raiz.derecho == null) return 1;  
4     return contarHojas(raiz.izquierdo) + contarHojas(raiz.derecho);  
5 }
```

Listing 5: Ejercicio 4 — Conteo de hojas (Java)

Capturas de ejecución — Ejercicio 4:



```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetail
ExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.jav
dt_ws\jdt.ls-java-project\bin' 'ejercicio3'
--- EJERCICIO 3 ---
Arbol Estructurado:
10
|--> 5
|   |--> 2
|   |   |--> 1
|   |   \--> 3
|   \--> 7
\--> 15
    |--> 12
    \--> 20
        |--> 18
        \--> 25
-----
La cantidad total de nodos es: 11
```

Figura 7: Ejercicio 4 — Ejecución en C++: árbol visualizado y conteo de nodos hoja

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> cd "c:\Users\maugu\Downloads\" ; if ($?) { g++ ejercicio4.cpp -o ejer
icio4 } ; if ($?) { .\ejercicio4 }
--- EJERCICIO 4 ---
Arbol Estructurado:
10
|--> 5
|   |--> 2
|   |   |--> 1
|   |   \--> 3
|   \--> 7
\--> 15
    |--> 12
    \--> 20
        |--> 18
        \--> 25
-----
La cantidad total de hojas (nodos finales) es: 6
```

Figura 8: Ejercicio 4 — Ejecución en Java: árbol visualizado y conteo de nodos hoja

Ejercicio 5 — Aplicación: Sistema Web con árbol binario

Se modela la arquitectura de un sistema web real mediante un árbol binario de cadenas de texto: la raíz es *Sistema Web*, sus hijos directos son los módulos *Usuarios* e *Inventario*, y cada uno de estos tiene dos submódulos. Los tres recorridos clásicos se mapean a funcionalidades concretas del sistema: Preorden muestra el menú de navegación (de arriba hacia abajo), Postorden simula la carga de dependencias (de abajo hacia arriba) y BFS genera el mapa del sitio por niveles. El programa expone un menú interactivo en consola que permite ejecutar cada opción en tiempo de ejecución.

Un menú interactivo permite ejecutar los tres recorridos:

- **Opción 1 — Menú principal (Preorden):** muestra la jerarquía de módulos.
- **Opción 2 — Procesar módulos internos (Postorden):** simula la carga de dependencias de abajo hacia arriba.
- **Opción 3 — Mapa del sitio (BFS):** lista los módulos por nivel de profundidad.

Construcción del árbol y menú en C++:



```
1  NodoWeb* raiz = crearNodo("Sistema Web");
2  raiz->izquierdo      = crearNodo("Usuarios");
3  raiz->derecho         = crearNodo("Inventario");
4  raiz->izquierdo->izquierdo = crearNodo("Registrar");
5  raiz->izquierdo->derecho  = crearNodo("Buscar");
6  raiz->derecho->izquierdo = crearNodo("Productos");
7  raiz->derecho->derecho   = crearNodo("Reportes");
8
9  do {
10     cout << "\n=== MENU INTERACTIVO: SISTEMA WEB ===" << endl;
11     cout << "1. Mostrar Menu Principal (Preorden)" << endl;
12     cout << "2. Procesar Modulos Internos (Postorden)" << endl;
13     cout << "3. Mostrar Mapa del Sitio (BFS)" << endl;
14     cout << "4. Salir" << endl;
15     cout << "Ingrese una opcion: ";
16     cin >> opcion;
17     // ... switch con llamadas a cada recorrido ...
18 } while (opcion != 4);
```

Listing 6: Ejercicio 5 — Sistema Web con árbol binario (C++)

Capturas de ejecución — Ejercicio 5 en C++:

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsI
ExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.j
dt_ws\jdt.ls-java-project\bin' 'ejercicio4'
--- EJERCICIO 4 ---
Arbol Estructurado:
10
|--> 5
|   |--> 2
|   |   |--> 1
|   |   \--> 3
|   \--> 7
\--> 15
    |--> 12
    \--> 20
        |--> 18
        \--> 25
```

Figura 9: Ejercicio 5 C++ — Menú interactivo del sistema web

```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsI
ExceptionMessages' '-cp' 'C:\Users\maugu\AppData\Roaming\Code\User\workspaceStorage\8e43c860ecee6cc9573e16b53e9a7c83\redhat.j
dt_ws\jdt.ls-java-project\bin' 'ejercicio4'
--- EJERCICIO 4 ---
Arbol Estructurado:
10
|--> 5
|   |--> 2
|   |   |--> 1
|   |   \--> 3
|   \--> 7
\--> 15
    |--> 12
    \--> 20
        |--> 18
        \--> 25
-----
La cantidad total de hojas es: 6
```

Figura 10: Ejercicio 5 C++ — Opción 1: Menú principal (Recorrido Preorden)



```
PS C:\Users\maugu\OneDrive\Desktop\Lab1_Redes_Completo (2)> cd "c:\Users\maugu\Downloads\" ; if ($?) { g++ ejercicio5.cpp -o ejercicio5 } ; if ($?) { .\ejercicio5 }

=== MENU INTERACTIVO: SISTEMA WEB ===
1. Mostrar Menu Principal (Preorden)
2. Procesar Modulos Internos (Postorden)
3. Mostrar Mapa del Sitio (BFS)
4. Salir
Ingrese una opcion: 1

-----
=== MENU PRINCIPAL ===
- Sistema Web
  - Usuarios
    - Registrar
    - Buscar
  - Inventario
    - Productos
    - Reportes
```

Figura 11: Ejercicio 5 C++ — Opción 3: Mapa del sitio (Recorrido BFS)

Capturas de ejecución — Ejercicio 5 en Java:

```
4. Salir
Ingrese una opcion: 2

-----

=== PROCESANDO MODULOS ===
Cargando dependencias de: [Registrar
Cargando dependencias de: [Buscar]
Cargando dependencias de: [Usuarios]
Cargando dependencias de: [Productos
Cargando dependencias de: [Reportes]
```

Figura 12: Ejercicio 5 Java — Menú interactivo del sistema web



```
Ingrese una opcion: 2

-----

=== PROCESANDO MODULOS ===
Cargando dependencias de: [Registrar]
Cargando dependencias de: [Buscar]
Cargando dependencias de: [Usuarios]
Cargando dependencias de: [Productos]
Cargando dependencias de: [Reportes]
Cargando dependencias de: [Inventario]
Cargando dependencias de: [Sistema Web]
-----
```

Figura 13: Ejercicio 5 Java — Opción 2: Procesar módulos internos (Recorrido Postorden)

```
Ingrese una opcion: 3

-----

=== MAPA DEL SITIO ===
Profundidad 0: Sistema Web |
Profundidad 1: Usuarios | Inventario |
Profundidad 2: Registrar | Buscar | Productos | Reportes |
-----
```

Figura 14: Ejercicio 5 Java — Opción 3: Mapa del sitio (Recorrido BFS)

2.8. Habilidades blandas empleadas en la práctica

- ☒ Pensamiento crítico
- ☒ Trabajo en equipo
- ☒ Responsabilidad
- ☒ Adaptabilidad
- ☐ Liderazgo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Flexibilidad
- ☐ La resolución de conflictos



2.9. Conclusiones

- Los Árboles Binarios de Búsqueda permiten organizar datos de manera jerárquica y eficiente; la propiedad de orden (nodo izquierdo < raíz < nodo derecho) garantiza que el recorrido *inorden* produzca siempre la secuencia ordenada de los valores almacenados.
- La implementación paralela en C++ y Java evidenció que, aunque la lógica algorítmica es idéntica en ambos lenguajes, la gestión de memoria difiere significativamente: C++ requiere administración manual de punteros, mientras que Java delega esta responsabilidad al recolector de basura.
- El recorrido BFS (Búsqueda por Amplitud), apoyado en una cola auxiliar, resultó fundamental para explorar el árbol nivel por nivel, lo que permite visualizar la estructura completa del árbol de forma intuitiva y es directamente aplicable a la representación de mapas de sitio web.
- El Ejercicio 5 demostró que las estructuras de árbol binario no son exclusivas del ámbito matemático, sino que modelan de manera natural jerarquías presentes en sistemas reales como menús de aplicaciones, directorios de archivos o módulos de un sistema web.
- Las funciones recursivas de conteo (*contarNodos* y *contarHojas*) ilustraron el poder de la recursividad para resolver problemas sobre estructuras de datos no lineales con código conciso y legible.

2.10. Recomendaciones

- Se recomienda liberar la memoria dinámica al finalizar el programa en C++ utilizando una función recursiva de eliminación (**delete**), especialmente en aplicaciones de larga ejecución, para evitar fugas de memoria.
- Para conjuntos de datos grandes o con muchos valores repetidos, se sugiere explorar variantes autobalanceadas como los Árboles AVL o Árboles Rojo-Negro, ya que un ABB no balanceado puede degenerar en una lista enlazada con complejidad $O(n)$ en el peor caso.
- Es conveniente extender los ejercicios implementando la operación de **eliminación de nodos** en el ABB, considerando los tres casos posibles (nodo hoja, nodo con un hijo y nodo con dos hijos), para completar el ciclo de vida de la estructura.
- Se recomienda visualizar los árboles con herramientas gráficas (por ejemplo, dibujándolos en JavaFX o con librerías Python como **matplotlib** + **networkx**) para facilitar la comprensión visual de su estructura durante el aprendizaje.

2.11. Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.^a ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] R. Sedgewick y K. Wayne, *Algorithms*, 4.^a ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [3] B. Stroustrup, *The C++ Programming Language*, 4.^a ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.
- [4] J. Bloch, *Effective Java*, 3.^a ed. Boston, MA, USA: Addison-Wesley, 2018.



2.12. Anexos

<https://github.com/Sebastian141005/Tarea2.2-Clonacion>